

AAAC++ 学习助手技术报告

基于 PyQt 的 C++ 面向对象编程交互式学习系统

小组成员：徐来、徐鹤鸣、丁思溢

2026 年 6 月 25 日

目录

1	项目概述	3
2	小组成员与分工	3
3	系统架构	3
3.1	整体架构	3
3.2	模块依赖关系	3
4	核心技术实现	4
4.1	跨 Qt 绑定兼容层	4
4.2	枚举兼容性处理	4
4.3	HTML 内容渲染引擎	5
4.4	自适应高度浏览器	5
4.5	学习进度持久化	5
4.6	交互式练习系统	5
5	课程内容设计	5
5.1	内容结构	5
5.2	数据模型设计	6
6	UI/UX 设计	6
6.1	视觉风格	6
6.2	交互设计	6
7	关键技术亮点	6
7.1	零样板代码的 Qt 绑定兼容方案	6
7.2	HTML 驱动的动态内容渲染	7
7.3	自适应高度控件	7
7.4	错误处理与状态管理	7
8	项目总结与反思	7

1 项目概述

AAAC++ 学习助手是一个基于 PyQt 框架的桌面应用，面向 C++ 初学者提供交互式的面向对象编程学习体验。系统以纯 Python 实现，借助 HTML 渲染引擎动态呈现教学内容，并集成了学习进度追踪、交互式练习、章节导航等功能。

本文从系统架构、核心技术实现、课程内容设计、UI/UX 设计、关键技术亮点等维度，对该项目进行技术分析。

2 小组成员与分工

本项目由三位成员协作完成，具体分工如下：

- **徐来**：负责程序主体框架的搭建，包括课程数据模型设计、核心交互逻辑实现、HTML 渲染引擎开发等主要功能模块的编写。
- **丁思溢**：负责程序主体功能的实现，包括学习进度持久化模块、交互式练习系统、侧边导航与页面切换逻辑等核心组件的开发。
- **徐鹤鸣**：负责对程序代码进行润色与优化，完善代码注释与类型注解，撰写技术报告，以及项目后续工作的整理与交付。

3 系统架构

3.1 整体架构

系统遵循 MVC (Model-View-Controller) 架构模式，将数据模型、用户界面和业务逻辑分离：

- **Model 层**：Chapter、Page、Module、Practice 等数据类定义课程内容的数据结构；ProgressStore 负责学习进度的持久化存储。
- **View 层**：基于 PyQt 的 QWidget 体系构建，包括 ChapterList（学习总览）、KnowledgePage（知识页面）、ModuleCard（内容卡片）等组件。
- **Controller 层**：MainWindow 作为核心控制器，协调导航、页面切换、进度更新等交互逻辑。

3.2 模块依赖关系

```
1 main.py
2 |-- build_curriculum() -> Chapter[]
3 |-- ProgressStore
```

```

4 |-- MainWindow
5   |-- ChapterList
6   |-- KnowledgePage
7       |-- ModuleCard
8           |-- AutoHeightBrowser
9           |-- Practice
10  |-- QTreeWidget
11 |-- QSS

```

Listing 1: 系统模块依赖关系（简化）

4 核心技术实现

4.1 跨 Qt 绑定兼容层

不同开发环境中可能安装不同的 Qt Python 绑定（PySide6、PyQt6 或 PyQt5）。系统通过一套兼容层解决了这一问题：

```

1 QT_WIDGETS = (
2     "QApplication QFrame QHBoxLayout QLabel QMainWindow"
3     " QMessageBox QPushButton QScrollArea QStackedWidget"
4     " QTextBrowser QTreeWidget QTreeWidgetItem QVBoxLayout"
5     " QWidget"
6 ).split()
7
8 for binding in ("PySide6", "PyQt6", "PyQt5"):
9     try:
10         core = import_module(f"{binding}.QtCore")
11         widgets = import_module(f"{binding}.QtWidgets")
12         Qt = core.Qt
13         globals().update(...)
14         break
15 except ImportError:
16     pass

```

Listing 2: 跨 Qt 绑定兼容层实现

该方案使用 `importlib.import_module` 依次尝试导入 `PySide6`、`PyQt6`、`PyQt5`，一旦成功，就将 `Qt` 模块和 `QtWidgets` 模块添加到 `globals()` 中。

4.2 枚举兼容性处理

```

1 def qt_enum(owner, name, group):
2     value = getattr(owner, name, None)

```

```

3     return value if value is not None else getattr(getattr(owner
        , group), name)

```

Listing 3: Qt 枚举兼容函数

该函数先尝试直接获取枚举值（PyQt5 风格），失败则通过分组名获取（PySide6/PyQt6 风格），确保代码在不同 Qt 绑定间无缝迁移。

4.3 HTML 内容渲染引擎

系统采用 QTextBrowser 配合动态生成的 HTML 渲染教学内容。代码模块中，通过字符串查找将 C++ 双斜线注释（//）渲染为绿色；文本模块自动将空行分隔的段落包装为 <p> 标签；escape() 函数防止 HTML 注入，确保特殊字符正确显示。

4.4 自适应高度浏览器

AutoHeightBrowser 通过重写 resizeEvent 和提供 update_height

4.5 学习进度持久化

进度存储采用 JSON 格式，ProgressStore 类负责读写 progress.json 文件。json.dumps(ensure_ascii = False, indent = 2) 将 Python 字典转换为 JSON 字符串，load() 将 JSON 字符串转换为 Python 字典。

4.6 交互式练习系统

练习题模块实现了可切换显示/隐藏答案的交互机制。使用 QPushButton 的 toggled 信号同时控制答案标签的可见性和按钮文本的切换，交互逻辑简洁。

5 课程内容设计

5.1 内容结构

课程内容采用五章递进式设计：

1. **类是什么**：介绍面向对象编程的核心概念，通过 Person 类案例建立直观认识。
2. **类的通用模板**：讲解成员变量、成员函数、构造函数、析构函数、友元、访问控制等六个核心知识点。
3. **Person 类案例**：综合运用前述知识点构建完整的 Person 类，包含高考身份核验的友元函数案例。
4. **继承、派生与多态**：讲解三种继承方式（public/protected/private）的访问权限变化，以及虚函数实现的运行时多态。

5. **Student 类案例**：构建 Person → Student → Undergraduate/Graduate 的多层继承体系，演示多态的实际应用。

5.2 数据模型设计

课程数据模型采用嵌套的 dataclass 结构：Chapter、Page、Module、Practice。@dataclass 装饰器大幅减少了样板代码。ModuleType 枚举区分文本、代码、练习三种内容类型，便于渲染引擎分流处理。completable 属性允许某些页面（如总览模板页）不计入完成度统计。

6 UI/UX 设计

6.1 视觉风格

系统采用扁平化设计语言，通过 QSS 实现统一的视觉风格：

- **配色方案**：主色调为深蓝灰色（#23323c），辅以白色卡片背景（#ffffff）和浅灰页面背景（#f3f5f7）。
- **圆角设计**：卡片、按钮、导航项均采用圆角（10px–18px）。
- **状态颜色**：完成状态使用绿色，学习中状态使用黄色，未完成状态使用灰色。
- **字体**：中文使用微软雅黑，代码使用 Cascadia Mono / Consolas 等宽字体。

6.2 交互设计

- **侧边导航树**：使用 QTreeWidgetItem 构建，支持章节折叠/展开，选中项高亮显示。
- **学习总览**：ChapterList 页面以卡片形式展示所有章节和小节的完成状态，支持一键跳转。
- **底部导航**：每个知识页面底部提供”上一节”/”下一节”按钮和”我学会了”切换按钮。
- **进度重置**：提供”重置学习状态”按钮，经确认对话框确认后清空所有进度。

7 关键技术亮点

7.1 零样板代码的 Qt 绑定兼容方案

跨 Qt 绑定兼容是项目中最具特色的设计之一。传统上，支持多个 Qt 绑定需要大量条件编译代码或多个分支。本项目通过动态导入和全局注入，使整个代码库无需条件判断即可兼容三种 Qt 绑定，显著降低了维护成本。

7.2 HTML 驱动的动态内容渲染

QTextBrowser 配合动态 HTML 的方案替代了传统的 Qt Widget 渲染。课程数据以纯文本或代码形式存储,渲染逻辑集中在 `text_html()` 和 `code_html()` 中。

7.3 自适应高度控件

AutoHeightBrowser 通过动态计算文档高度并设置固定高度,解决了 QTextBrowser 在布局中无法自适应高度的问题。内容卡片可以自然地嵌入垂直布局,无需嵌套滚动条。

7.4 错误处理与状态管理

ProgressStore.load() 包含对 JSON 解析错误的处理,确保程序不会因数据文件损坏而崩溃。侧边导航使用 `syncing` 标志位防止回调循环,确保导航状态与页面状态的一致性。

8 项目总结与反思

AAAC++ 学习助手作为一个面向 C++ 初学者的交互式学习工具,在设计和实现上取得了一定的成果。系统采用 MVC 架构,模块划分清晰,各组件职责明确,便于后续维护和扩展。跨 Qt 绑定的兼容方案使得程序能够在不同环境下运行,HTML 驱动的渲染方式为内容呈现提供了较大的灵活性。

在开发过程中,我们也遇到了一些值得反思的问题。首先,课程内容目前以硬编码方式写在程序中,后续若需扩充内容,维护成本较高,可以考虑将课程数据以 JSON 或 Markdown 文件的形式外部化存储。其次,交互式练习目前仅支持选择题,功能较为单一,未来可以加入填空题、代码填空题等更多题型。此外,程序缺少代码运行环境,学习者无法在应用内直接运行 C++ 代码进行实践,这是一个较为明显的功能缺失。

总体而言,本项目在有限的时间内完成了核心功能的开发,基本达到了预期的学习辅助目标。后续可以从内容外部化、题型多样化、集成在线编译器等方面进行改进,使工具更加完善和实用。